# DriveShare – Peer-to-Peer Car Rental Platform
# Design Patterns Implementation

By: Hakan Turk

# System Overview

Features:

- User registration & login
- Car listing & management
- Search & booking system
- Messaging system
- Notifications
- Payment simulation

Tech Stack:

- Flask
- SQLite

# Architecture

Architecture Layers:

- Presentation Layer to Flask routes + HTML
- Service Layer to Business logic
- Pattern Layer to Design patterns
- Data Layer to SQLite database

# DESIGN PATTERNS USED

Builder

Singleton

Observer

Mediator

Proxy

Chain of Responsibility

# Builder Pattern

In the standard Builder pattern, you have a Builder and a Product.
In my system:

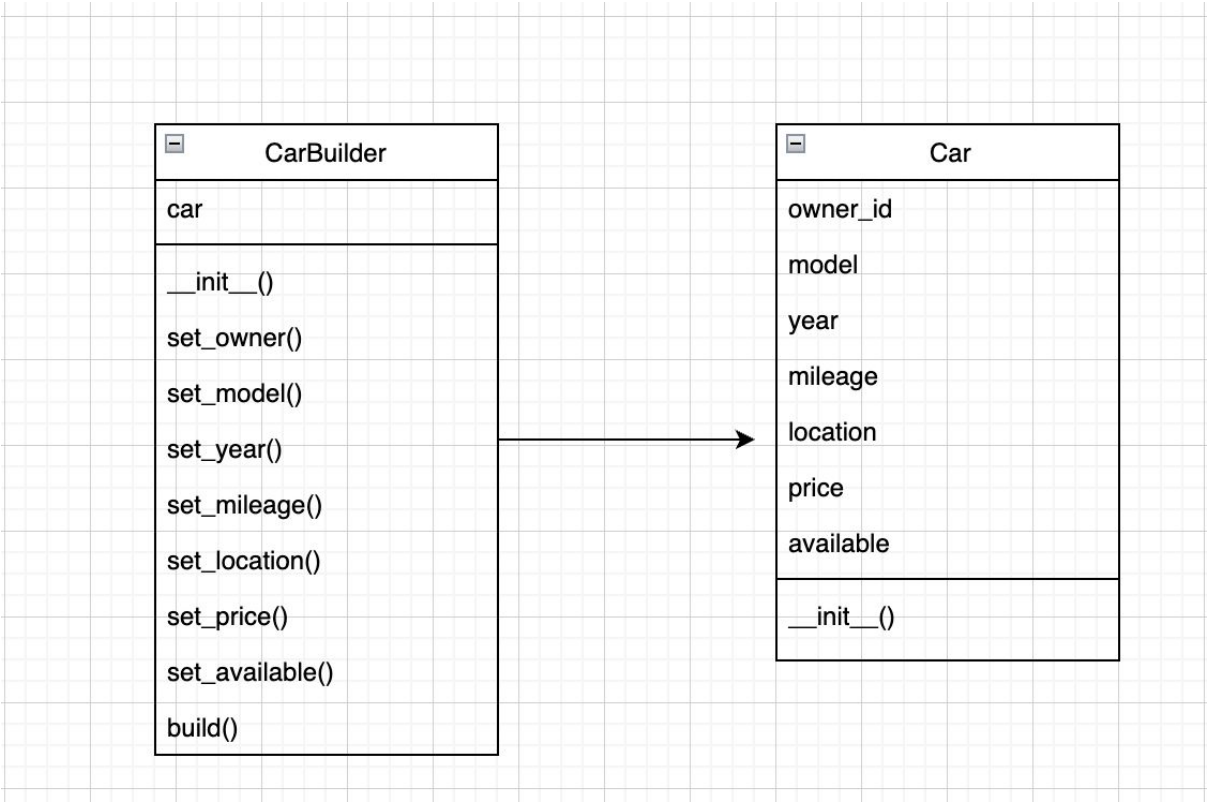
- Car = Product
- CarBuilder = Builder

Implementation:

- CarBuilder builds Car objects step-by-step
- Used inside CarService

Purpose:

- Cleaner object creation
- Supports optional attributes
- Improves readability



# Singleton Pattern

In the standard Singleton pattern, only one instance of a class exists.

In my system:

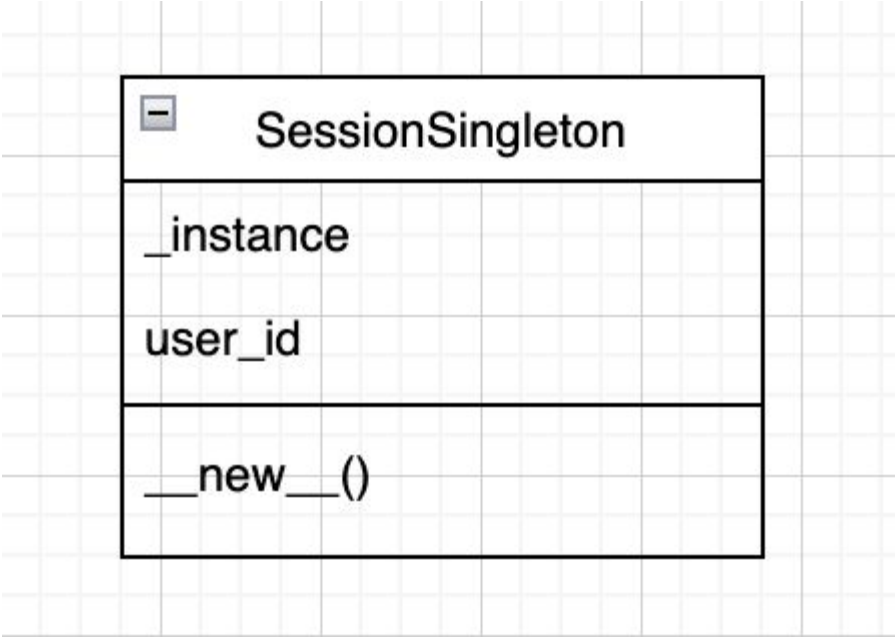
- SessionSingleton ensures only one session object is used

Implementation:

- Stores current user_id

Purpose:

- Maintains global session state
- Prevents multiple instances



# Observer Pattern

In the standard Observer pattern:

- Subject manages observers
- Observers react to updates

In my system:

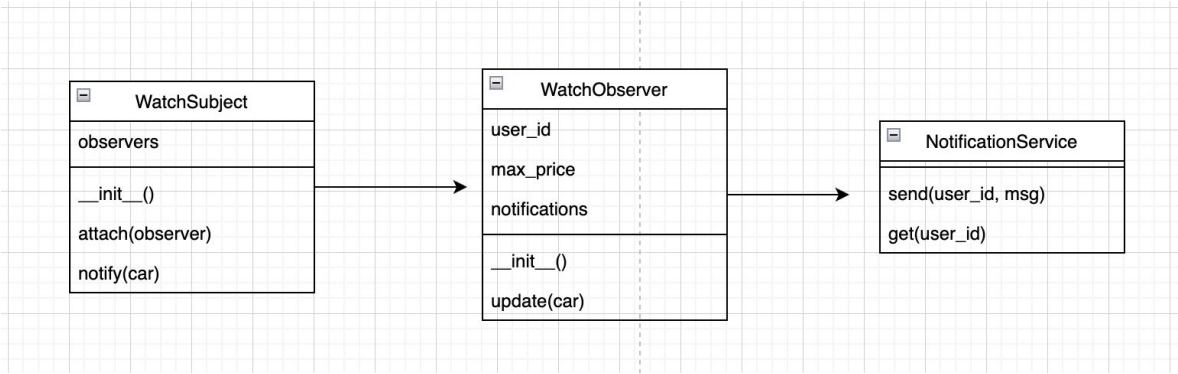
- WatchSubject = Subject
- WatchObserver = Observer

Triggers:

- Price drop
- Availability change

Purpose:

- Real-time notifications
- Users can “watch” cars



# Mediator Pattern

In the standard Mediator pattern:

- Mediator controls communication
- Colleagues communicate through it

In my system:

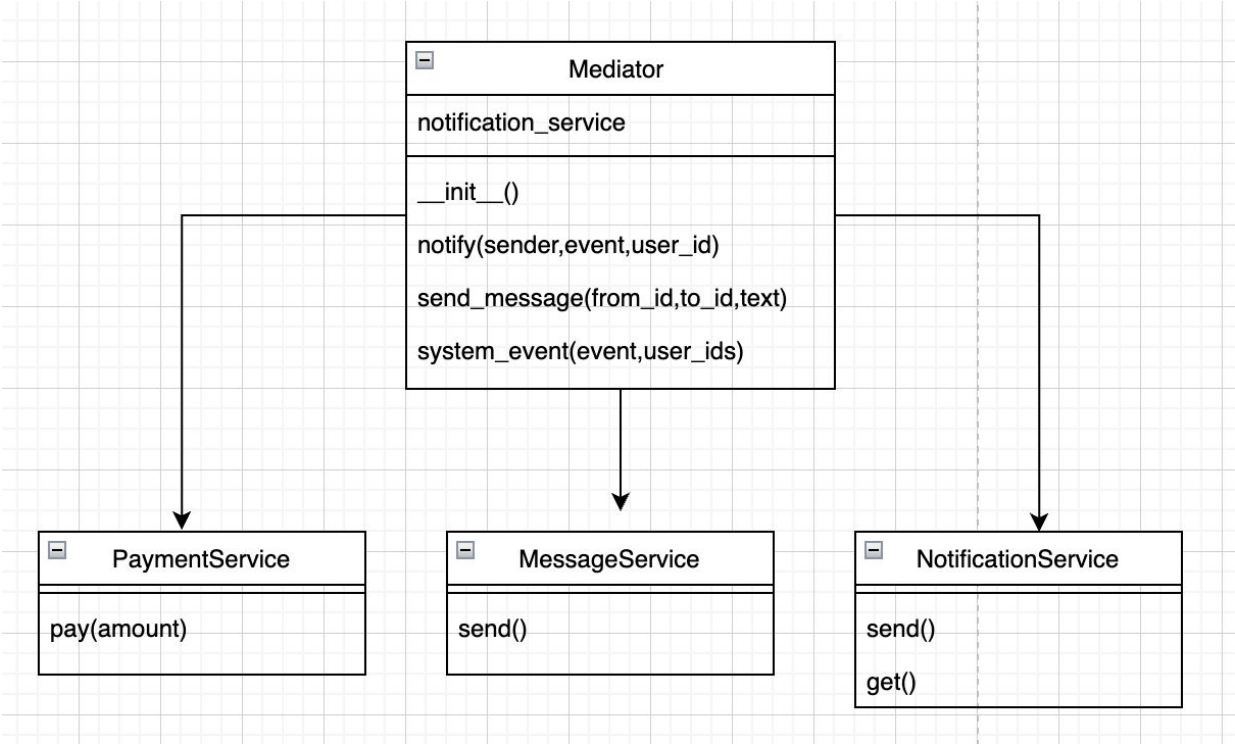
- Mediator = central class
- Colleagues = Payment, messaging, system events

Handles:

- Messaging
- Payment notifications
- System updates

Purpose:

- Reduces coupling
- Centralizes communication



# Proxy Pattern

In the standard Proxy pattern:

- Real Subject does the actual work
- Proxy controls access

In our system:

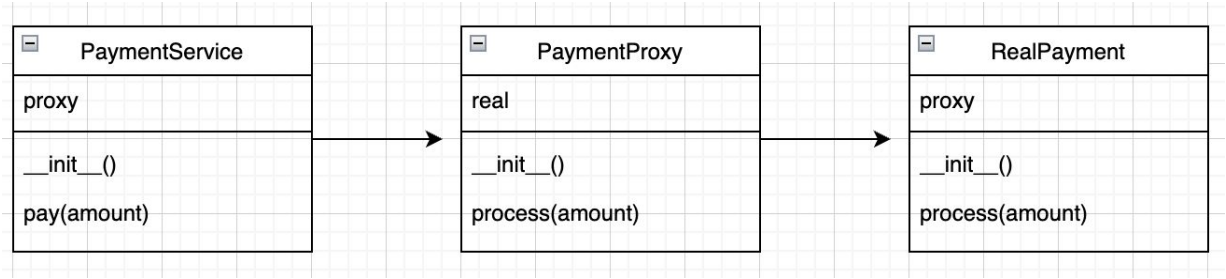
- RealPayment = Real Subject
- PaymentProxy = Proxy

Function:

- Validates payment
- Calls real payment

Purpose:

- Adds security layer
- Controls access



# Chain Pattern

In the standard Chain pattern:

- Requests pass through multiple handlers

In my system:

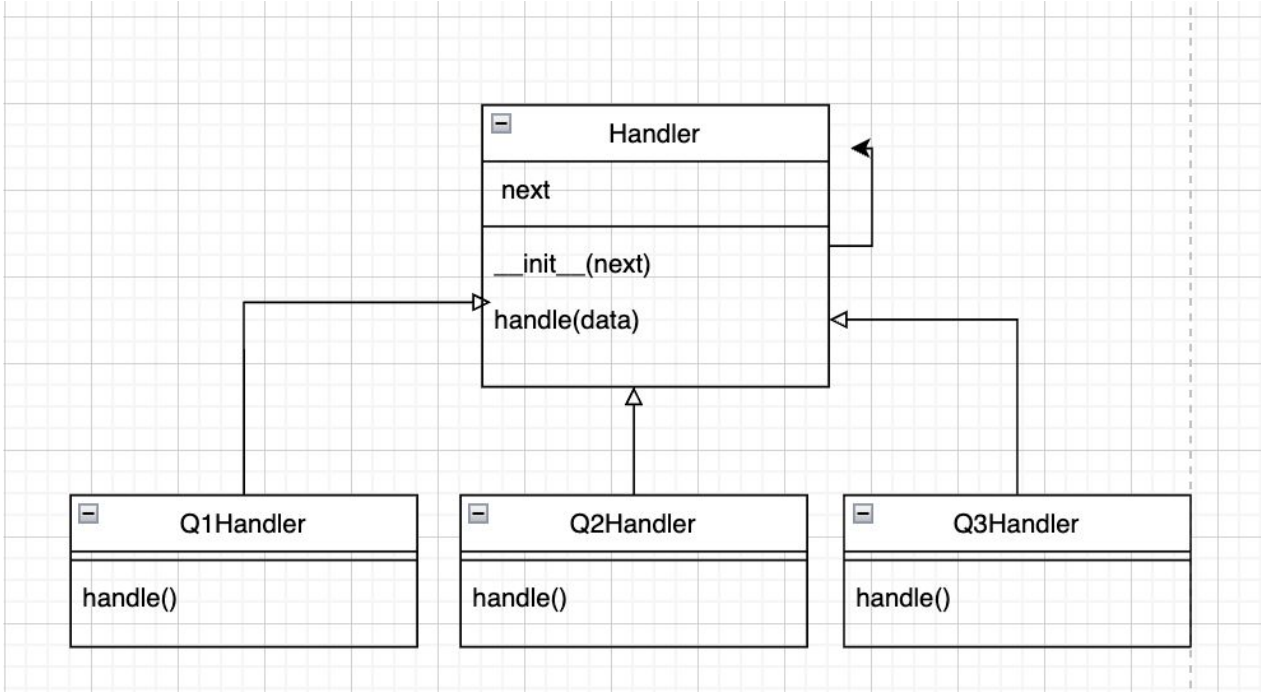
- Q1Handler then Q2Handler then Q3Handler

Function:

- Validates each security question step-by-step
- Stops immediately if any answer is incorrect
- Only allows password reset if all checks pass

Purpose:

- Multi-step verification
- Secure password recovery



# Conclusion

Summary:

- Builder to object creation
- Singleton to session management
- Observer to notifications
- Mediator to communication
- Proxy to payment security
- Chain to authentication